



Regex 2

Taller 2

Tomas Parra

Teoría de lenguajes y laboratorio

Profesora: Luz Páez

Medellín, 2026

Facultad de Ingeniería

Universidad de Antioquia

1. Análisis de formulario

Campo	Información solicitada	Regla
Nombre	Nombre del participante	Solo letras y espacios
Apellido	Apellido del participante	Solo letras y espacio
Correo E.	Correo para el registro	Debe tener formato de correo
Teléfono	Número telefónico	Solo números, entre 7 y 10 dígitos
Documento	Número de identificación	Solo números, entre 6 y 10 dígitos

2. Campo: Nombre

Información solicitada

El nombre de la persona que desea registrarse.

Regla

- Debe contener solamente letras.
- Puede contener espacios.
- No se permiten números.
- No se permiten símbolos.
- No puede estar vacío.

Ejemplos válidos

- Carlos
- Juan David
- Maria
- Ana Sofia

Ejemplos inválidos

- Carlos123
- Juan@
- Pedro_
- 12345

Expresión regular

$\wedge[A-Za-zÁÉÍÓÚáéíóúÑñ]+([A-Za-zÁÉÍÓÚáéíóúÑñ]+)*\$$

3. Campo: Apellido

Información solicitada

El apellido de la persona.

Regla

- Solo letras.
- Puede tener más de una palabra.
- No permite números.
- No permite símbolos.
- No puede estar vacío.

Ejemplos válidos

- Perez
- Gomez
- Garcia Lopez
- Rodriguez

Ejemplos inválidos

- Perez123
- Gomez@
- 123Perez
- Lopez_

Expresión regular

$^[A-Za-zÁÉÍÓÚáéíóúÑñ]+([A-Za-zÁÉÍÓÚáéíóúÑñ]+)*\$$

4. Campo: Correo electrónico

Información solicitada

Correo electrónico del participante.

Regla

El correo debe tener:

- Una parte antes del @.
- El símbolo @.
- Un dominio.
- Un punto en el dominio.
- No debe contener espacios.

Ejemplos válidos

- juan@gmail.com
- maria.perez@hotmail.com
- estudiante@udea.edu.co

Ejemplos inválidos

- juan@gmail
- juan.com
- @gmail.com
- juan @gmail.com
- juan@gmail

Expresión regular

$^[A-Za-z0-9._\%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$$

5. Campo: Teléfono

Información solicitada

Número telefónico del participante.

Regla

- Debe contener solamente números.
- Debe tener entre 7 y 10 dígitos.
- No se permiten letras.
- No se permiten espacios ni símbolos.

Ejemplos válidos

- 3001234567
- 6041234567
- 1234567

Ejemplos inválidos

- 300123
- 300123456789
- 300ABC4567
- 300-123-4567

Expresión regular

$^{[0-9]\{7,10\}}\$$

6. Campo: Documento

Información solicitada

Número de documento de identificación.

Regla

- Solo números.
- Entre 6 y 10 dígitos.
- No debe contener letras.
- No debe contener espacios ni símbolos.

Ejemplos válidos

- 123456789
- 1002345678
- 1234567

Ejemplos inválidos

- ABC123456
- 12345
- 12345678901
- 123-456-789

Expresión regular

$^{[0-9]\{6,10\}}\$$

7. Programa

Codigo

```
1. #include <iostream>
2. #include <string>
3. #include <regex>
4.
5. using namespace std;
6.
7. int main()
8. {
9.     string nombre;
10.    string apellido;
11.    string correo;
12.    string telefono;
13.    string documento;
14.
15.    // Expresiones regulares
16.    regex nombreRegex(
17.        "[A-Za-z]+( [A-Za-z]+)*$"
18.    );
19.
20.    regex correoRegex(
21.        "[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
22.    );
23.
24.    regex telefonoRegex(
25.        "[0-9]{7,10}$"
26.    );
27.
28.    regex documentoRegex(
29.        "[0-9]{6,10}$"
30.    );
31.
32.    cout << "===== " << endl;
33.    cout << "          REGISTRO AL EVENTO" << endl;
34.    cout << "===== " << endl;
35.    cout << endl;
36.
37.    // NOMBRE
38.    do
39.    {
40.        cout << "Nombre: ";
41.        getline(cin, nombre);
42.
```

```
43.     if (!regex_match(nombre, nombreRegex))
44.     {
45.         cout << "Nombre invalido. Intente nuevamente." << endl;
46.     }
47.
48. } while (!regex_match(nombre, nombreRegex));
49.
50. cout << "Nombre valido." << endl;
51. cout << endl;
52.
53. // APELLIDO
54. do
55. {
56.     cout << "Apellido: ";
57.     getline(cin, apellido);
58.
59.     if (!regex_match(apellido, nombreRegex))
60.     {
61.         cout << "Apellido invalido. Intente nuevamente." << endl;
62.     }
63.
64. } while (!regex_match(apellido, nombreRegex));
65.
66. cout << "Apellido valido." << endl;
67. cout << endl;
68.
69. // CORREO
70. do
71. {
72.     cout << "Correo electronico: ";
73.     getline(cin, correo);
74.
75.     if (!regex_match(correo, correoRegex))
76.     {
77.         cout << "Correo invalido. Intente nuevamente." << endl;
78.     }
79.
80. } while (!regex_match(correo, correoRegex));
81.
82. cout << "Correo valido." << endl;
83. cout << endl;
84.
85. // TELEFONO
86. do
87. {
```

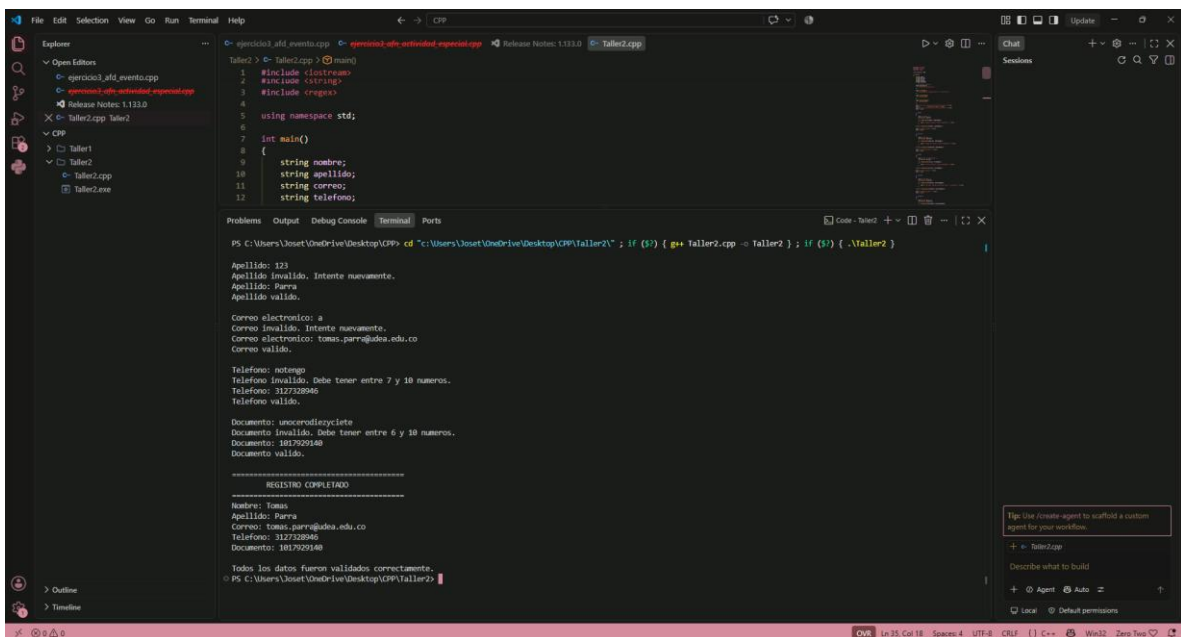
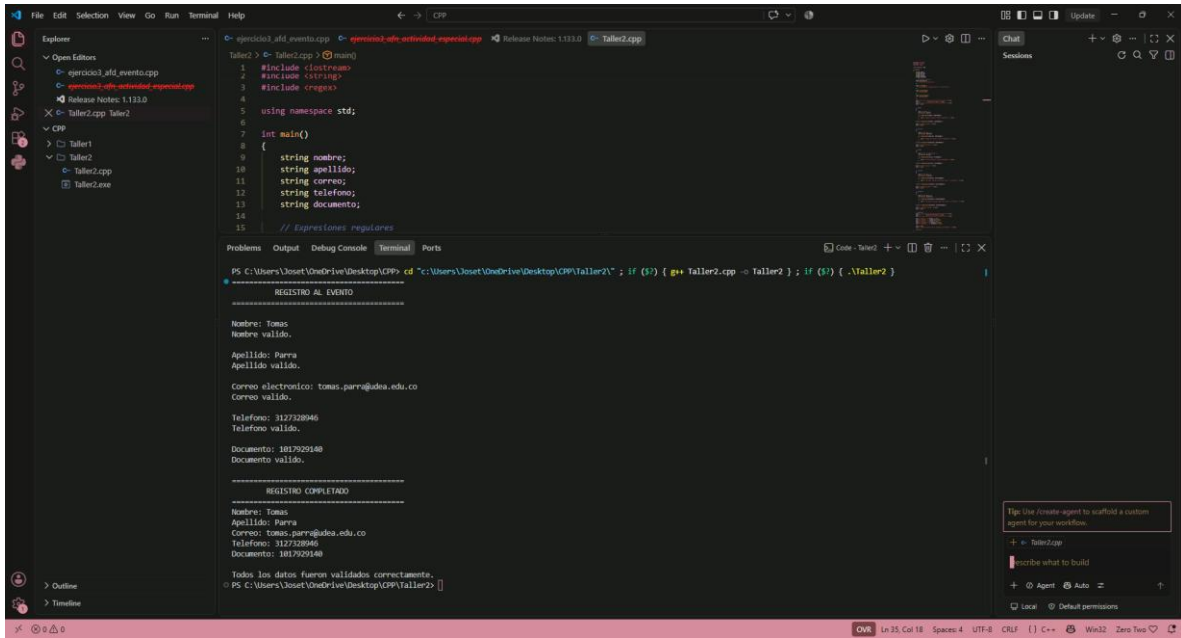
```

88.         cout << "Telefono: ";
89.         getline(cin, telefono);
90.
91.         if (!regex_match(telefono, telefonoRegex))
92.         {
93.             cout << "Telefono invalido. Debe tener entre 7 y 10
numeros." << endl;
94.         }
95.
96.     } while (!regex_match(telefono, telefonoRegex));
97.
98.     cout << "Telefono valido." << endl;
99.     cout << endl;
100.
101.         // DOCUMENTO
102.         do
103.         {
104.             cout << "Documento: ";
105.             getline(cin, documento);
106.
107.             if (!regex_match(documento, documentoRegex))
108.             {
109.                 cout << "Documento invalido. Debe tener entre 6 y 10
numeros." << endl;
110.             }
111.
112.         } while (!regex_match(documento, documentoRegex));
113.
114.         cout << "Documento valido." << endl;
115.         cout << endl;
116.
117.         // RESULTADO
118.         cout << "===== " << endl;
119.         cout << "             REGISTRO COMPLETADO" << endl;
120.         cout << "===== " << endl;
121.
122.         cout << "Nombre: " << nombre << endl;
123.         cout << "Apellido: " << apellido << endl;
124.         cout << "Correo: " << correo << endl;
125.         cout << "Telefono: " << telefono << endl;
126.         cout << "Documento: " << documento << endl;
127.
128.         cout << endl;
129.         cout << "Todos los datos fueron validados correctamente." <<
endl;

```

```
130.
131.     return 0;
132. }
133.
```

8.Pruebas



9. Conclusiones

El programa permite simular la validación de un formulario mediante expresiones regulares en C++. Se utilizaron diferentes patrones para comprobar que los nombres y apellidos contengan únicamente letras, que el correo tenga una estructura válida y que los números de teléfono y documento contengan únicamente la cantidad permitida de dígitos.

Con esto se demuestra el uso de las expresiones regulares para validar información ingresada por un usuario y evitar que el formulario continúe mientras existan datos que no cumplen las reglas establecidas.